

MongoDB for PageIndex Retrieval: Matched Benchmark and Schema Evaluation

Junyao Dong

Abstract. PageIndex stores documents as large hierarchical JSON trees that an agent navigates from the root to relevant nodes. This report benchmarks storage-level operations corresponding to ConDB retrieval on MongoDB, PostgreSQL/JSONB, DuckDB, and SQLite, at scales up to ten million nodes. Schema effects and engine effects are measured separately. In a matched three-million-node experiment, MongoDB and PostgreSQL scan the same 200 subtree paths, return the same node ids, and use equivalent covering indexes. On the monolithic layout, MongoDB is $4.35\times$ slower at P50 and $3.91\times$ at P95; on narrow structure stores, it is $4.65\times$ and $4.07\times$ slower. A MongoDB-specific ten-million-node ablation shows that a lean covering index eliminates document fetches and reaches 15.5 ms P50 / 168 ms P95. Together, these results support covering the scan while leaving the full structure/metadata/text split to an end-to-end deployment test.

1 Introduction

ConDB indexes a document as a PageIndex tree: a hierarchy of nodes carrying a title, a summary, page indices, and (at the leaves) text. Retrieval is a top-down traversal: the retriever starts at the root, lists a node’s children to decide where to descend, and fetches the content of selected nodes. ConDB also exposes subtree access for tree formatting and utility paths. This report treats its ids-only range scan as a storage stress microbenchmark rather than as a replay of the complete retriever. Section 3 presents the results, the scan diagnosis, and the candidate schema; Section 4 specifies its deployment validation.

The operation set follows ConDB’s storage interface. Generic database operations outside that interface (ancestor-to-root walks, page-range filters, naive substring scans) are excluded.

2 Workload and methodology

Operations. Table 1 maps the measured storage primitives to the corresponding ConDB retrieval operations. The `get_subtree` microbenchmark is an ids-only materialized-path range scan. Its latency excludes root lookup, metadata resolution, tree formatting, and pagination. The report also measures storage footprint, the write path, and concurrency.

Data. The datasets are synthetic PageIndex trees in the canonical format, validated against ConDB’s `DocumentTreeAdapter`, at two scales (Table 2). Random-word content preserves the target payload sizes; the benchmark focuses on tree shape and per-node size.

Table 1: Storage primitives represented in the benchmark and their corresponding ConDB uses.

Operation	Storage call	Used for
Point lookup	<code>get_node(tree_id, node_id)</code>	Navigate to / read one node
Expand children	<code>get_children(tree_id, node_id)</code>	List children, pick where to descend
Get subtree	<code>get_subtree(tree_id, node_id, depth)</code>	Tree formatting and utility paths
Fetch content	<code>get_entity(tree_id, node_id)</code>	Read selected nodes’ content

Table 2: Datasets.

Dataset	Nodes	JSON size	Depth	Fan-out
Medium	70,843	85 MB	5	6–12
Large	10,000,000	14.06 GB	8	6–14

Engines and setup. MongoDB 7.0 (Community Edition) and PostgreSQL 16 (JSONB document column) run as servers in Docker over localhost; DuckDB and SQLite are embedded. The medium/large suite runs on a 96-core Linux host with 1 TB of RAM, using the same monolithic one-record-per-node layout for cross-engine rows. The relational engines flatten the tree to one row per node. Each read uses fixed seeded samples; every query is checked to return the same row count. Each concurrency level runs its client processes for five seconds. This host has ample memory, so reads are cache-resident rather than disk-bound.

Subtree test sets. The matched 3M experiment is the cross-engine scan comparison: a clean 16-vCPU AMD EPYC 9K65 host with 30 GiB RAM, a three-million-node tree, and one engine per invocation. MongoDB and PostgreSQL receive the same 200 seeded paths (average result: 8,840.4 ids) under three matched configurations: *naïve*, a monolithic store with a plain `path` index; *covered*, the same store with an equivalent covering index; and *narrow structure*, a separate structure relation with the same covering index. PostgreSQL uses `(path) INCLUDE (node_id)` followed by `VACUUM ANALYZE` so `EXPLAIN` confirms a true index-only scan; MongoDB uses `{path,node_id}`, excludes `_id` from projection, and confirms `PROJECTION_COVERED` with no `FETCH`. The 10M MongoDB component measurements in Section 3.3 cover the larger tree. Unless stated otherwise, subtree timings refer to the ids-only scan phase.

Execution model and comparability. SQLite and DuckDB execute in process, whereas MongoDB and PostgreSQL include localhost protocol overhead. MongoDB–PostgreSQL is therefore the server-to-server latency comparison; the embedded-engine measurements are contextual references. SQLite runs the same three subtree configurations. DuckDB’s columnar projection already reads only `node_id`, so the benchmark does not define independent covered or narrow-structure measurements for it. Storage figures likewise reflect each engine’s native encoding and compression: MongoDB’s `storageSize` is post-compression, DuckDB uses columnar encoding, and PostgreSQL may compress out-of-line values. MongoDB’s logical BSON size is reported separately. Each comparison is single-host within its own test series. The operations through concurrency use one tree per database; multi-tenancy, batched reads, and deletion use 25 co-resident trees.

3 Experiments and diagnosis

3.1 Comparative benchmark

3.1.1 Storage and ingest

Table 3: Storage and ingest (large dataset). On-disk totals include the indexes. MongoDB holds 15.21 GB of logical BSON in 5.27 GB of data on disk (WiredTiger snappy).

Engine	On-disk	Index	Uncompr.	Ingest (n/s)	Build
DuckDB	5.02 GB	–	–	348,831	10 s
MongoDB	5.81 GB	536 MB	15.21 GB	41,387	69 s
PostgreSQL (JSONB)	17.71 GB	1.19 GB	–	62,816	52 s
SQLite	19.51 GB	980 MB	–	52,880	38 s

In the reported on-disk footprints (Table 3), DuckDB benefits from columnar encoding and MongoDB from WiredTiger compression. DuckDB’s index bytes are included in its database file but were not measured separately.

3.1.2 Retrieval operations

The navigation reads (point lookup, children, content fetch) are sub-millisecond on MongoDB, PostgreSQL, and SQLite; DuckDB is slower on these point-oriented operations. MongoDB is roughly $3\times$ slower than PostgreSQL per call (Table 4), while the absolute difference remains below one millisecond. The multi-record `get_subtree` scan is reported separately because its result depends strongly on index coverage. In the matched 3M experiment, with equivalent covering indexes, MongoDB is $4.35\times$ slower than PostgreSQL at P50 and $3.91\times$ at P95. Section 3.2 isolates the MongoDB `FETCH` cost, and Section 3.3 carries the test to 10M nodes on MongoDB.

Table 4: Single-record retrieval latency on the common monolithic layout (ms; lower is better). Best per row in bold. Subtree ID-scan results appear in Table 5.

Operation	MongoDB	PostgreSQL	DuckDB	SQLite
<i>Medium, P50</i>				
Point lookup (<code>get_node</code>)	0.238	0.081	2.705	0.008
Expand children (<code>get_children</code>)	0.251	0.090	0.856	0.017
Fetch content (<code>get_entity</code>)	0.228	0.071	2.750	0.007
<i>Large, P50 / P95</i>				
Point lookup	0.270 / 0.32	0.087 / 0.09	5.59 / 8.97	0.012 / 0.014
Expand children	0.282 / 0.32	0.099 / 0.14	4.28 / 5.05	0.023 / 0.032
Fetch content	0.249 / 0.32	0.081 / 0.09	5.21 / 8.35	0.012 / 0.013

Equivalent covering indexes isolate the engine path on the monolithic store; the narrow-structure configuration then measures the effect of applying the same physical split to both engines. MongoDB remains about $4\times$ slower than PostgreSQL in both configurations. These results cover a cache-warm ID scan; metadata resolution and tree formatting are evaluated separately.

Table 5: Matched ids-only subtree matrix: 3M nodes, the same 200 paths, average 8,840.4 returned ids, one client, cache-warm (P50 / P95 ms; lower is better). Every row compares the same configuration. The ratio is MongoDB latency divided by PostgreSQL latency.

Matched configuration	MongoDB	PostgreSQL	MongoDB / PostgreSQL
Monolith + plain path index	2.391 / 19.201	0.648 / 6.817	3.69 / 2.82×
Monolith + covering index	1.291 / 10.692	0.297 / 2.732	4.35 / 3.91×
Narrow structure + covering index	1.269 / 10.616	0.273 / 2.610	4.65 / 4.07×

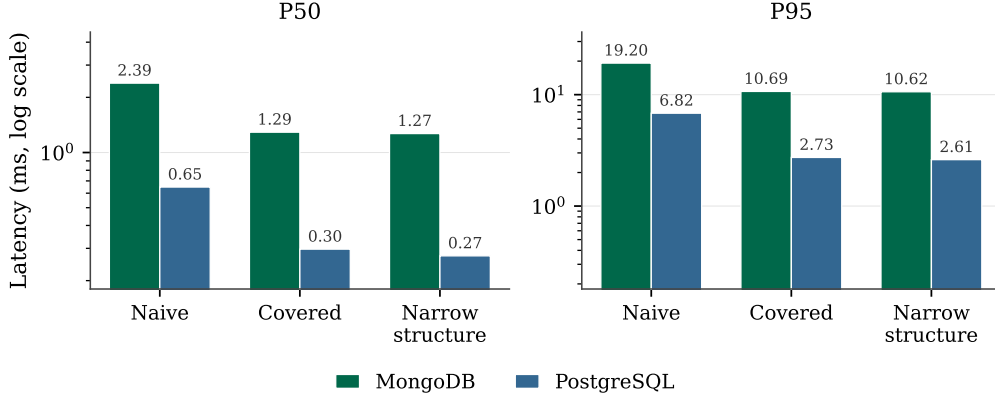


Figure 1: Latency from the matched 3M experiment in Table 5 (log scale). Each pair uses the same layout and index capability.

3.1.3 Write path

PostgreSQL records lower update latency under the tested settings (Table 6). Because durability settings differ, these figures are descriptive rather than a throughput ranking. The reported storage deltas cover only this 5,000-update run.

Table 6: Write path (medium): 5,000 field updates, 2,000 incremental inserts. “Bloat” is observed on-disk growth.

Engine	Upd. P50	Upd. ops/s	Insert ops/s	Bloat
MongoDB	0.242	4,050	5,586	0 MB
PostgreSQL (JSONB)	0.123	7,856	8,732	7 MB
DuckDB	2.966	332	450	7 MB
SQLite	0.017	40,442	21,815	0 MB

3.1.4 Concurrency

In the recorded five-second point-lookup run (Figure 2), MongoDB dips from one to two clients and then rises through 64; PostgreSQL’s highest recorded throughput is also at 64, while SQLite peaks at eight and then declines. Because each level ran once and worker measurement windows were not start-synchronized, the figure describes the scaling trend rather than a precise saturation point.

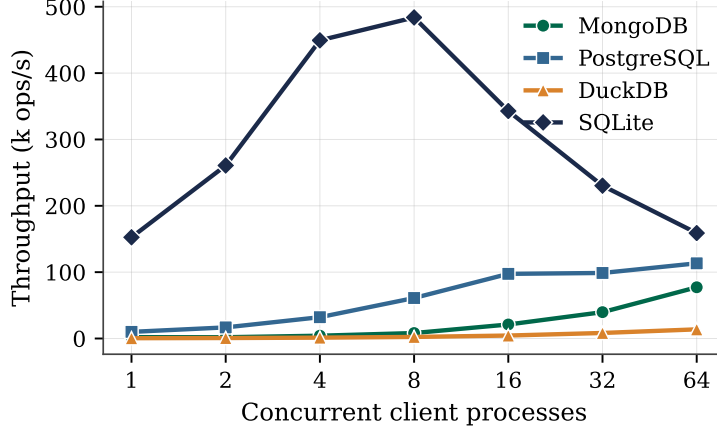


Figure 2: Point-lookup throughput versus concurrent client processes (medium).

3.1.5 Multi-tenancy, batched reads, and deletion

Three additional deployment-relevant operations are evaluated on 25 co-resident copies of the medium tree (1,771,075 rows): tenant-filtered reads, batched candidate lookup, and whole-tree deletion. Every read is filtered by `tree_id` and served by a compound index led by `tree_id`.

Multi-tenant selectivity. With 25 trees in one collection, a `tree_id`-led index keeps MongoDB point and children latency within about $1.5\times$ of the corresponding single-tree measurements (Tables 4 and 7). This experiment does not characterize larger tenant counts.

Table 7: Per-query reads with 25 trees co-resident in one store, each filtered by `tree_id` (P50, ms).

Operation	MongoDB	PostgreSQL	DuckDB	SQLite
Point lookup	0.345	0.124	3.772	0.008
Expand children	0.372	0.147	3.073	0.014

Batched reads. The navigation step returns a list of candidate node ids, so the retriever fetches several nodes at once. One batched read (`$in` on MongoDB, `IN` on the relational engines) beats the point-lookup loop it replaces on every engine, and the margin grows with batch size because each iteration of the loop on a server engine is a round trip. At 200 ids MongoDB is 4.6 ms batched against 273 ms looped, a $60\times$ gap; PostgreSQL is 1.4 ms against 29 ms (Table 8). MongoDB gains the most because batching amortizes the per-call client overhead that costs it on single reads; in-process SQLite, with no round trip, gains about $2\times$. The retriever should resolve a candidate set with one batched read, not a loop.

Deletion. Removing a tree (one tenant, 70,843 of 1,771,075 rows) is a lifecycle operation, not part of retrieval. The delete is fast on every engine, under four seconds, but none returns disk space on the delete alone (Table 9). SQLite (`VACUUM`) and PostgreSQL (`VACUUM FULL`) reclaim the removed tree’s footprint through a full, locking rewrite that takes 10 to 17 s. In this run, MongoDB `compact` and DuckDB checkpoint do not measurably shrink the file after a single-tenant deletion.

Table 8: Batched id-list read at 10, 50, and 200 ids against the 200-id point-lookup loop it replaces (P50, ms per batch).

Engine	Batched (n ids)			Loop
	10	50	200	200
MongoDB	0.46	1.56	4.56	273
PostgreSQL (JSONB)	0.27	0.78	1.40	29
DuckDB	19.6	22.2	39.1	790
SQLite	0.03	0.13	0.50	0.97

The benchmark therefore supports the measured delete rates, but not a general claim about space reclamation under repeated deletion cycles.

Table 9: Whole-tree delete and space reclamation (delete one of 25 trees).

Engine	Delete (rows/s)	Reclaim	Reclaim time	On-disk (before → after)
PostgreSQL (JSONB)	157,578	VACUUM FULL	17.3 s	2.77 → 2.66 GB
SQLite	141,842	VACUUM	10.6 s	2.56 → 2.46 GB
MongoDB	60,680	compact	0.1 s	911 MB, no shrink
DuckDB	20,171	checkpoint	–	790 MB, no shrink

Together, these measurements support `tree_id`-led lookup indexes and batched candidate reads; whole-tree deletion remains a lifecycle operation.

3.2 MongoDB subtree ID-scan diagnosis

MongoDB’s unit of read is the whole document. A secondary index (here on `path`) is a separate B-tree holding only the indexed field and a record id; to return any other field, the engine must *fetch* the full BSON document and apply the projection to it. A point lookup is one such fetch and is cheap. A subtree scan over monolithic node documents—`structure`, `title`, `summary`, and leaf `text` in one document—is tens of thousands of them, each reading and decoding the whole document no matter how small the projection. The observed `FETCH` plan and `totalDocsExamined` count confirm that `path`. With resident data and no cache evictions, the measured cost is per-document work—locate the record, materialize the document, project, stream through the cursor—times subtree size. In the monolithic benchmark, the other primitives touch few documents and remain fast.

Two controls isolate the cause on the ten-million-node tree (one measuring client, resident collection, identical 200 seeded paths). Running 16, 32, or 64 background point-lookup processes leaves scan latency almost unchanged, with no cache evictions. Removing the wide `text` field cuts each scanned document by ~75% but also leaves latency unchanged; even ~150-byte topology-only documents retain most of the cost. The dominant variable is the number of fetches, which the candidate schema removes from the scan path (Section 3.3).

Row and column stores avoid this per-document work on the ID scan. A row store materializes only the requested columns (PostgreSQL leaves the JSONB datum compressed, since `node_id` is its own column), and a column store reads only the requested column, so both do far less per-node work on this one scan. The monolithic measurements show inexpensive point, children, and content reads. After metadata is decoupled, their complete cost may include an additional point or batched

read and remains part of the end-to-end schema test. The candidate layout below isolates topology so the large ID scan can stay in the index.

3.3 Candidate schema: decoupled structure and key-value metadata

A node carries three kinds of data with different access patterns. The candidate schema stores them separately:

- **Structure** (`node_id`, `path`, `parent_id`, `depth`): ~ 150 -byte topology documents carrying a lean `{tree_id, path, node_id}` covering index. Serves the subtree ID scan and topology navigation.
- **Metadata** (`title`, `summary`): a key-value collection keyed by `(tree_id, node_id)` (or an equivalently namespaced `_id`). Serves rendered nodes by point or batched read.
- **Text**: leaf body text in its own collection under the same tenant-qualified key. Serves `get_entity` for the few nodes the model finally selects.

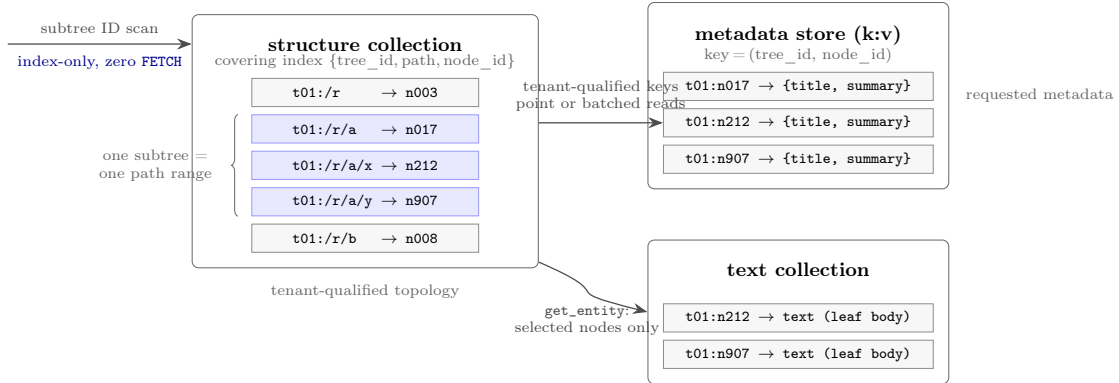


Figure 3: Candidate schema. A contiguous range of the `{tree_id, path, node_id}` covering index returns tenant-qualified node keys; those keys address metadata and text.

In the single-tree 10M MongoDB component test, the covered ID scan examines no documents: `explain` reports `PROJECTION_COVERED` and `totalDocsExamined: 0`, with 15.5 ms P50 / 168 ms P95 (Table 10). Covered execution has two operational requirements. First, the index must contain every projected field and the projection must exclude `_id`; otherwise the per-document `FETCH` returns. Second, the covering index should be built after bulk load so the write-once tree pays its maintenance cost once. Without coverage, even the ~ 150 -byte structure documents run about $1.6\times$ slower.

Table 10: MongoDB component latency on the 10M tree (one client, ms). The metadata row reads every node of a $\sim 36k$ -node subtree in fixed 1,000-id batches.

Read	P50	P95	Note
Structure scan (covered, ids)	15.5	168	<code>totalDocsExamined: 0</code>
Structure scan without covering index	25.2	260	<code>FETCH</code> per node
+ metadata for all nodes, 1k-id batches	96.3	1034	fixed batching: ~ 36 requests

The ids-only scan excludes the cost of rendering a tree view. The metadata row resolves *every* node of a large subtree using fixed 1,000-id batches and costs ~ 1 s. Its ~ 36 requests reflect that batch size rather than MongoDB’s 16 MiB command limit; with this dataset’s short ids, the complete BSON command is about 0.62 MiB. End-to-end latency should therefore be measured with the formatter’s actual fields, batching, and result bounds. If a deployment does need the full `node_id+title+summary` view from a single indexed query, the alternative is to carry the metadata inside the covering index of a merged collection: it stays covered and beats the monolithic scan, but the multi-kilobyte `summary` key inflates that index to roughly the size of the data itself, which is why the candidate schema keeps metadata in a key-value store instead. Keep the anchored materialized-path range as the predicate either way—an unanchored regex cannot form a range and must scan the whole index.

The measured component footprints are also smaller. For ten million nodes, the single-tree structure collection occupies 0.28 GB of data plus 0.57 GB of index, while the metadata store occupies 0.69 GB plus a 0.14 GB `_id` index. This ~ 1.7 GB navigation footprint compares with 5.3 GB for the monolithic collection with inline `text`. The metadata measurement uses `node_id` as its single-tree key, so the tenant-qualified index requires its own sizing. Whether the smaller footprint improves cache residency under mixed navigation and content reads is measured by the cache-constrained test in Section 4.

Unmeasured structural extensions. Nested Sets or a tree-order `_id` on a clustered collection turns the subtree into one range query or a sequential read; a pre-computed subtree document turns it into a point read. A write-once tree avoids steady-state renumbering or rewrite costs, but these designs add ingest work and storage amplification. Their value depends on the complete retrieval path and must be measured before adding schema complexity.

4 Deployment validation plan

The structure scan and metadata stress case in Section 3.3 were measured on a cache-resident host; the complete three-store path was not. A production deployment also adds the opposite constraint: many trees or tenants can make the working set larger than RAM, so the plan must also keep the hot navigation data in cache. The candidate validation plan is therefore:

1. **Validate the decoupled schema end to end.** Structure (`tree_id`, `node_id`, `parent_id`, `depth`, `path`) in the navigation collection; `title` and `summary` in a key-value metadata collection keyed by (`tree_id`, `node_id`); body text in a third collection under the same key. Table 10 measures the covered structure component. The complete path, including tenant-qualified keys, metadata resolution, and formatting, requires an end-to-end measurement.
2. **Lead tenant-local query indexes with `tree_id`.** Point, children, subtree, and content reads should use compound indexes beginning with `tree_id`, preserving selectivity as the number of trees grows. A shard key may use the same prefix, but its final form must also account for tenant size and write distribution.
3. **Keep the covering index lean and exactly aligned with the projection.** A `{tree_id,path,node_id}` index serves subtree ID scans index-only; build it after bulk load so its maintenance cost is paid once. Covered execution is all-or-nothing—a missing projected field, or `_id` left in the projection, silently brings the per-document `FETCH` back—so treat `explain` showing `totalDocsExamined: 0` as a deployment check, not a one-time observation. Do not put the multi-kilobyte `summary` into the covering key: the metadata store is intended

to serve it separately, and carrying it in the index costs index bytes comparable to the data itself.

4. **Denormalize hot subtree views only after measuring demand.** If endpoint profiling identifies subtree rendering as a top latency contributor, use write-once structure to precompute bounded structure+summary subtree views, or add Nested Sets / tree-order `_id` for range-locality. These remove even the covered scan’s per-key work, but they add storage amplification and more complicated ingest.

The required cache test compares the decoupled schema with a monolithic inline-text layout while sizing the WiredTiger cache below the monolithic working set and above the decoupled navigation working set. This directly measures the residency benefit that a fully cache-resident run cannot show.

5 Conclusion

The results separate the value of the index from the remaining engine gap. In the matched 3M experiment, covering improves MongoDB from 2.391/19.201 ms to 1.291/10.692 ms P50/P95. PostgreSQL with equivalent coverage reaches 0.297/2.732 ms, making MongoDB $4.35\times$ slower at P50 and $3.91\times$ at P95. Moving the covered scan to a narrow structure store adds little warm-cache benefit on either engine; the corresponding MongoDB/PostgreSQL ratio is $4.65\times/4.07\times$. These figures describe the ids-only scan, not the complete retrieval endpoint.

MongoDB’s document read path explains the optimization: a non-covered scan pays per matched document regardless of projection, whereas the covering index keeps the ID scan in the B-tree. Decoupling structure from metadata and text may also reduce the navigation footprint, but that benefit depends on the complete access pattern. The 10M MongoDB result of 15.5 ms/168 ms provides the larger-scale capacity point. A deployment decision still requires the cache-constrained layout comparison and a complete, tenant-qualified structure/metadata/text measurement using the formatter’s real fields and result bounds.